Proyecto 2

Arquitectura de Computadores · Universidad de los Andes · Semestre 2026-2 · Profesor: Jorge Gómez Mir

Integrantes: José Tomás Arévalo · Santiago Arenas · Danna Collazos

1. Resumen

El proyecto instancia el Pochoco SoC (procesador Espino RV32E, RAM y periféricos) en la FPGA Lattice iCE40 HX1K de la Nandland Go Board y ejecuta sobre él un juego de reflejos escrito en assembly. El programa `sw/game.s` se traduce a `sw/game.hex` con un ensamblador propio en Python (`asm/asm.py`), que no invoca ningún ensamblador RISC-V externo.

Los cambios al hardware se limitan a dos: un contador de ciclos de 32 bits legible en `0x8000_000C` y el módulo superior `rtl/game_top.v`. El juego ocupa 184 palabras (736 bytes de los 2048 disponibles) y el diseño sintetizado usa cerca de 1180 de 1280 celdas lógicas (92 %) con cierre de temporización a 25 MHz.

.

2. Arquitectura, contador de ciclos y temporización de tres segundos

2.1 Visión general

El diseño sintetizado es `game_top`, que instancia `pochoco_soc` con el parámetro `MemFile = "sw/game.hex"` y conecta los puertos de la Go Board (reloj, 4 LEDs, 4 botones, 2 displays de 7 segmentos y el SPI, este último sin uso). `game_top` no agrega lógica propia: su función es seleccionar la imagen de memoria del juego. El procesador, la RAM y la interfaz SPI se mantienen como en el repositorio base.

El programa accede a los periféricos por memoria mapeada, con base `0x8000_0000`:

Dirección	Offset	Acceso	Función
`0x8000_0000`	`0x00`	escritura	byte mostrado en los displays (un dígito hexadecimal por display)
`0x8000_0004`	`0x04`	escritura	LEDs: el bit *i* controla el LED *i*
`0x8000_0008`	`0x08`	lectura	botones: el bit *i* es 1 si el botón *i* está apretado
`0x8000_000C`	`0x0C`	lectura	contador de ciclos (agregado por el grupo)

El botón *i* se asocia al LED *i*: superior izquierdo = 0, inferior izquierdo = 1, superior derecho = 2, inferior derecho = 3.

2.2 Modificaciones al RTL

En `rtl/pochoco_periph.v` se agregó el registro `cycle_q` de 32 bits, que suma 1 en cada flanco de subida del reloj y se reinicia con el reset asíncrono. Se expone como offset 3 (`addr_i[7:2] = 3`) en el multiplexor de lectura, junto a los botones. Es de solo lectura: una escritura a ese offset no tiene efecto.

```
reg [31:0] cycle_q;
always @(posedge clk_i or negedge rst_ni)
  if (!rst_ni) cycle_q <= 32'b0;
  else         cycle_q <= cycle_q + 32'd1;
// lectura:  6'd3: rdata_o <= cycle_q;
```

El núcleo Espino no se modificó. El costo en recursos fue pasar de 1082 a cerca de 1180 celdas lógicas de 1280 (de 84 % a 92 %), con una frecuencia máxima estimada de unos 40 MHz frente a los 25 MHz del reloj de la placa.

2.3 Unidad de tiempo y conversión desde ciclos

El reloj de la Go Board es de 25 MHz, por lo que el contador avanza una cuenta cada 40 ns. El tiempo se muestra en décimas de segundo, y la conversión está parametrizada en `game.s` mediante `FRECUENCIA` y `CICLOS_DECIMA`:

Magnitud	Valor
Frecuencia de reloj	25.000.000 Hz
Periodo de un ciclo	40 ns
1 décima de segundo (`CICLOS_DECIMA = FRECUENCIA / 10`)	2.500.000 ciclos
3 segundos	75.000.000 ciclos
Tope mostrado, 9,9 s (`CICLOS_TOPE`)	247.500.000 ciclos
Desborde del contador (2^{32} ciclos)	171,8 s

Como RV32E no tiene división, la conversión se hace en la subrutina `ciclos_a_decimas` restando `CICLOS_DECIMA` mientras el valor alcance y contando las restas. El resultado se trunca, como un cronómetro: 2,38 s se muestra como `23`.

2.4 Temporización de los tres segundos

La espera se implementa en la subrutina `esperar`, que recibe en `a0` la cantidad de ciclos:

```
esperar:
   lw   t0, CONTADOR(s0)     # inicio
esp_1:
   lw   t1, CONTADOR(s0)
   sub  t1, t1, t0          # ciclos transcurridos
```

```
    bltu t1, a0, esp_1      # repetir mientras no se cumpla el plazo
    ret
```

Para la fase inicial se carga a0 = CICLOS_INICIO = DECIMAS_INICIO * CICLOS_DECIMA = 30 × 2.500.000 = 75.000.000 y se llama a esperar con los cuatro LEDs encendidos. Esa constante no cabe en un inmediato de 12 bits, por lo que el ensamblador la carga con lui + addi (0x04787 y -1856).

Tres propiedades de este diseño importan para su validez:

Diferencias, no valores absolutos: La comparación usa ahora − inicio en aritmética sin signo de 32 bits, que es correcta aunque el contador dé la vuelta entre ambas lecturas, mientras el intervalo sea menor a 171,8 s.

Cota inferior garantizada: El bucle termina solo cuando se cumplen al menos 75.000.000 ciclos, por lo que la fase dura al menos 3 s. Inmediatamente después, soltar_botones espera como mínimo 20 ms adicionales con los LEDs aún encendidos, de modo que la fase real es algo mayor a 3,02 s.

Constante única: Los demás plazos (CICLOS_ERROR, CICLOS_FINAL, CICLOS_PARPADEO, CICLOS_REBOTE) se derivan de CICLOS_DECIMA, y el ensamblador evalúa las expresiones .equ. Cambiar DECIMAS_INICIO a 50 produce 125.000.000 ciclos sin tocar nada más.

2.5 Restricción del procesador: sin desplazamientos

El Espino del repositorio base tiene los shifts deshabilitados en la ALU (ALU_SLL, ALU_SRL y ALU_SRA están comentados «para ahorrar LUTs») y los ejecuta como suma. Reactivarlos con un barrel shifter de 32 bits costaría cientos de celdas lógicas, que no caben con el contador ya instalado (92 % de uso). Se decidió dejar la ALU intacta y resolver en software lo que habría requerido shifts:

-Desplazar a la izquierda 1 bit es add rd, rs, rs.

-Multiplicar por 16 es sumar cuatro veces (×2, ×4, ×8, ×16).

-Leer un bit del LFSR se hace con una máscara (and) y snez.

-Las divisiones por 2.500.000 y por 10 se hacen con restas sucesivas.

3. Diseño del juego

3.1 Secuencia

Etapa	LEDs	Display	Transición
Inicio (al programar)	apagados	00	cualquier botón: mezcla la semilla y comienza la partida
Fase inicial de cada ronda	los 4 encendidos	tiempo anterior	75.000.000 ciclos, luego se esperan botones sueltos

Etapa	LEDs	Display	Transición
Espera de respuesta	solo el LED objetivo	tiempo anterior	el contador se lee en la instrucción siguiente al encendido
Acierto	—	tiempo en décimas	suma al acumulador; si faltan rondas, nueva ronda
Botón incorrecto	apagados	`EE`	espera 1,0 s y repite la misma ronda; los aciertos se conservan
Tras la ronda 10	apagados	tiempo de la ronda 10	2,0 s, luego se calcula el promedio
Pantalla final	alternan `1001` / `0110`	promedio	cualquier botón inicia otra partida

3.2 Constantes configurables

Se definen al inicio de `game.s` con `.equ`:

Constante	Valor	Función
`FRECUENCIA`	25.000.000	reloj en Hz
`CICLOS_DECIMA`	`FRECUENCIA / 10`	ciclos por décima
`DECIMAS_INICIO`	30	fase con 4 LEDs (3,0 s)
`DECIMAS_ERROR`	10	duración del `EE` (1,0 s)
`DECIMAS_FINAL`	20	tiempo de la ronda 10 visible (2,0 s)
`DECIMAS_PARPADEO`	5	periodo del parpadeo final (0,5 s)
`DECIMAS_TOPE`	99	máximo mostrado (9,9 s)
`RONDAS`	10	aciertos por partida

3.3 Uso de registros

Registro	Uso
`s0`	base de periféricos `0x8000_0000`, fija durante todo el programa

Registro	Uso
s1	rondas correctas de la partida
gp	estado del generador pseudoaleatorio (LFSR de 32 bits)
tp	suma de los tiempos de rondas correctas, en décimas
sp	pila, parte en 0x800 (fin de la RAM) y crece hacia abajo
a0	argumento y resultado de subrutinas
a1…a5, t0…t2	temporales (a1 objetivo, a2 inicio de medición, a3 tope, a4 botones, a5 ciclos de reacción)

3.4 Generación pseudoaleatoria

La elección del LED combina una semilla dependiente del jugador con un generador determinista:

1. **Semilla.** La subrutina mezclar_semilla hace gp = gp XOR contador en cada pulsación (la de inicio, la de cada respuesta y la de reinicio). Como el instante en que una persona aprieta no se repite entre partidas, la secuencia tampoco. Si el resultado fuera 0 se fuerza a 1, porque un LFSR en 0 queda fijo.
2. **LFSR de Galois de 32 bits** con polinomio $x^{32} + x^{22} + x^2 + x + 1$ (0x00400007), de periodo máximo. Cada elección avanza 8 pasos: se duplica el estado (add gp, gp, gp) y, si el bit 31 era 1, se aplica xor con el polinomio.
3. **Índice** = 2·bit 17 + bit 16 del estado, leídos con máscara y snez. Una tabla .word 0b0001, 0b0010, 0b0100, 0b1000 convierte el índice en la máscara del LED.

Se usan los bits 16 y 17 porque el bucle que espera el botón lee el contador cada número par de ciclos, y el bit 0 de esa lectura casi no varía. Esta decisión se verificó con un modelo del generador de 200.000 rondas, en el que cada LED salió el 25 % de las veces.

La partida no comienza sola al energizar: el programa ejecuta siempre la misma secuencia de ciclos, así que sin una pulsación inicial la primera ronda sería idéntica en cada arranque. El botón de inicio aporta la primera semilla.

4. Código assembly, medición de tiempo y promedio

4.1 Estructura del programa

Etiqueta	Función
_start	inicializa pila, base de periféricos y LFSR; apaga LEDs; muestra 00

Etiqueta	Función
esperar_inicio	espera el primer botón; mezcla semilla; antirrebote
nueva_partida	pone en cero s1 y tp
ronda	fase de 4 LEDs, elección de objetivo y medición
esperar_boton	bucle de medición y detección de pulsación
ronda_error	muestra EE, apaga LEDs, espera y repite la ronda
fin_partida	muestra el tiempo 10, calcula el promedio y parpadea
esperar, esperar_o_boton	esperas por ciclos (la segunda aborta si hay botón)
soltar_botones	antirrebote por software (usa la pila para guardar ra)
mezclar_semilla, elegir_objetivo	generación pseudoaleatoria
ciclos_a_decimas, promedio, mostrar_decimas	aritmética y salida a displays

4.2 Instrucciones utilizadas

Aritmético-lógicas: add, sub, addi, xor, xori, and, or. Memoria: lw, sw. Control: beq/bne/blt/bltu/bge/bgeu y jal/jalr. Constantes y direcciones: lui, auipc. Pseudo-instrucciones: li, la, mv, call, ret, j, beqz, bnez, bgez, bleu, snez. No se usa ningún desplazamiento, multiplicación ni división.

4.3 Inicio de la medición

El requisito pide iniciar la medición «en el mismo instante lógico» en que comienza la ronda. El código lo resuelve escribiendo el LED y leyendo el contador en instrucciones consecutivas:

```
sw   a1, LEDS(s0)       # enciende solo el LED objetivo
lw   a2, CONTADOR(s0)   # a2 = inicio de la medición
```

En simulación del SoC completo entre ambos eventos pasan 1 ciclo (40 ns).

4.4 Bucle de medición

```
esperar_boton:
   lw   t1, CONTADOR(s0)
   sub  t2, t1, a2         # t2 = ciclos desde que se encendió el LED
   bltu t2, a3, leer_botones
   sub  a2, t1, a3         # tope alcanzado: el tiempo queda fijo en 9,9 s
leer_botones:
   lw   t0, BOTONES(s0)
   beqz t0, esperar_boton
```

El valor de t2 se calcula antes de leer los botones, así que el tiempo registrado nunca incluye la espera posterior a la pulsación. Según las simulaciones del grupo, cada vuelta del bucle dura 14 ciclos (560 ns), que es el máximo retraso de detección y es unas 180.000 veces menor que la unidad mostrada (0,1 s). Cuando el tiempo llega al tope, se corre a2 para mantener la diferencia en CICLOS_TOPE, de modo que el contador relativo nunca se acerca al desborde.

Tras la pulsación se guardan a4 = botones y a5 = ciclos, se mezcla la semilla y se compara a4 con el LED objetivo a1. Dos botones simultáneos producen un valor distinto al objetivo y cuentan como error.

4.5 Acierto, error y visualización

En un acierto, a5 se convierte a décimas con ciclos_a_decimas (acotado a 99), se suma a tp, se incrementa s1 y se muestra el valor. En un error se escribe 0xEE en el display, se apagan los LEDs, se espera CICLOS_ERROR y se salta a ronda sin modificar s1 ni tp, con lo que las rondas correctas acumuladas se conservan.

Como el periférico muestra el byte en hexadecimal (un dígito por display), mostrar_decimas separa decenas y unidades restando 10 y escribe decenas·16 + unidades: 23 décimas se envía como 0x23 y se ve «2 3». El producto por 16 se obtiene con cuatro sumas.

4.6 Promedio de diez rondas

El acumulador tp suma los valores en décimas que se mostraron en cada ronda. Al terminar la ronda RONDAS, promedio divide por RONDAS con restas sucesivas y redondea a la décima más cercana: si el doble del resto es al menos RONDAS, el cociente sube una unidad. Por ejemplo, una suma de 165 décimas da 16,5 y se muestra como 17. El promedio es, por lo tanto, el de los diez valores mostrados (ya truncados), y el valor máximo posible es 99.

4.7 Antirrebote

soltar_botones espera a que no haya botones apretados y a que sigan sueltos 20 ms (CICLOS_REBOTE = CICLOS_DECIMA / 5); si hay rebote, vuelve a esperar. Se usa antes de elegir el objetivo (para que un botón mantenido no termine la ronda al instante), tras cada acierto o error y al reiniciar la partida. Es la única subrutina que anida otra llamada, por lo que guarda ra en la pila (sw ra, 0(sp)).

5. Ensamblador

5.1 Uso

```
python asm/asm.py sw/game.s -o sw/game.hex          # genera el .hex
python asm/asm.py sw/game.s -o sw/game.hex -l build/game.lst   # y un listado
python asm/asm.py sw/game.s -o sw/game.hex -D RONDAS=5         # redefine una constante
```

La salida es game.hex, con una palabra de 32 bits en hexadecimal por línea. Para el juego, el ensamblador informa: 184 palabras (736 bytes de 2048), 34 etiquetas. Está escrito en Python estándar, sin bibliotecas externas.

5.2 Sintaxis aceptada

Elemento	Sintaxis
Comentarios	# hasta el fin de línea
Etiquetas	nombre:, solas o en la misma línea que una instrucción
Registros	x0–x15 y nombres ABI zero ra sp gp tp t0-t2 s0/fp s1 a0-a5
Inmediatos	decimal, 0x, 0b, negativos y expresiones con + - * / ( )
Directivas	.equ/.set NOMBRE, valor; .word v1, v2, …; .text, .section, .global (sin efecto)
Operandos de memoria	offset(registro), donde offset puede ser una constante .equ (por ejemplo LEDS(s0))

Instrucciones soportadas: todo RV32E base (add sub sll slt sltu xor srl sra or and, addi slti sltiu xori ori andi slli srli srai, lb lh lw lbu lhu, sb sh sw, beq bne blt bge bltu bgeu, lui auipc jal jalr, fence ecall ebreak) y las pseudo-instrucciones li la mv not neg seqz snez nop j jr call ret beqz bnez bltz bgez blez bgtz bgt ble bgtu bleu.

5.3 Algoritmo de dos pasadas y etiquetas

La **pasada 1** recorre el archivo, evalúa las constantes .equ, asigna una dirección a cada línea (4 bytes por palabra emitida) y registra cada etiqueta con la dirección en que aparece. Es necesaria porque un salto puede apuntar a una etiqueta definida más abajo, cuya dirección aún no se conoce al llegar a la instrucción. La **pasada 2** codifica cada instrucción ya con todas las etiquetas resueltas.

En saltos y ramas el campo inmediato es relativo al PC: destino − dirección de la instrucción. Las pseudo-instrucciones que ocupan más de una palabra deben decidir su tamaño en la pasada 1: li ocupa 1 palabra si el valor cabe en 12 bits con signo y 2 si no; la y call ocupan siempre 2 (auipc + addi y auipc + jalr, respectivamente).

5.4 Inmediatos y carga de constantes de 32 bits

Cuando li necesita 32 bits, el valor se parte en una parte alta (20 bits, para lui) y una baja (12 bits con signo, para addi). Como addi extiende el signo, si el bit 11 del valor es 1 la parte baja es negativa y la alta debe incrementarse en 1 para compensar. Para 75.000.000 = 0x047868C0:

- parte baja: 0x8C0 − 0x1000 = −1856
- parte alta: (0x047868C0 + 1856) >> 12 = 0x04787
- resultado: lui a0, 0x04787 seguido de addi a0, a0, −1856

5.5 Codificación por formato

Formato	Instrucciones	Disposición de bits (31 → 0)
R	add, sub, xor, and, or, …	funct7 · rs2 · rs1 · funct3 · rd · opcode
I	addi, xori, lw, jalr, …	imm[11:0] · rs1 · funct3 · rd · opcode
S	sw, sb, sh	imm[11:5] · rs2 · rs1 · funct3 · imm[4:0] · opcode
B	beq, bne, blt, bltu, …	imm[12|10:5] · rs2 · rs1 · funct3 · imm[4:1|11] · opcode
U	lui, auipc	imm[31:12] · rd · opcode
J	jal, j	imm[20|10:1|11|19:12] · rd · opcode

5.6 Ejemplo de traducción de assembly a código máquina

Tomados del listado generado con -l para game.s (dirección, palabra y línea fuente):

Dirección	Palabra	Fuente	Campos
0x0068	00b42223	sw a1, LEDS(s0)	S: imm[11:5]=0000000, rs2=a1 (01011), rs1=s0 (01000), funct3=010, imm[4:0]=00100 (offset 4), op=0100011
0x0044	04787537	li a0, CICLOS_INICIO	U: lui a0, 0x04787 (op 0110111)
0x0048	8c050513	(segunda palabra de li)	I: addi a0, a0, −1856 (imm 100011000000)
0x0080	00d3e463	bltu t2, a3, leer_botones	B: salto hacia adelante, offset +8 (0x88 − 0x80), funct3=110
0x008c	fe0286e3	beqz t0, esperar_boton	B: salto hacia atrás, offset −20 (0x78 − 0x8c), imm[12|10:5]=1|111111, imm[4:1|11]=0110|1

Dirección	Palabra	Fuente	Campos
0x0020 / 0x0024	00000097 / 1a8080e7	call mezclar_semilla	auipc ra, 0 + jalr ra, 0x1a8(ra): destino 0x20 + 0x1a8 = 0x1c8

5.7 Validación

El ensamblador no escribe el .hex si detecta cualquier error, y los informa todos juntos, cada uno con su número de línea y el texto de la línea. Valida:

- registros fuera de RV32E (x16, a6, …);
- rangos de inmediatos de 12 y 20 bits y de los desplazamientos (shamt);
- alcance de saltos condicionales (±4 KiB) y de jal;
- etiquetas repetidas o no definidas, y constantes .equ repetidas o que usan símbolos no definidos;
- instrucciones desconocidas o fuera de RV32E (por ejemplo mul) y cantidad incorrecta de operandos;
- operandos de memoria sin formato offset(registro);
- tamaño del programa respecto de la RAM (512 palabras).

Para las instrucciones de desplazamiento, que el Espino ejecuta como suma, el ensamblador emite una advertencia pero las codifica igualmente.

5.8 Pruebas

El script asm/pruebas/probar_asm.py ejecuta 32 casos en cuatro grupos, y al correrlo sobre el repositorio todos aprueban (32/32):

Grupo	Casos	Qué verifica
Referencias completas	4	blink.s, 7seg.s y buttons_leds.s producen exactamente los .hex oficiales del Pochoco; todas.s (96 palabras, todas las instrucciones) coincide con el ensamblador GNU de RISC-V, usado una sola vez para generar la referencia todas.hex
Codificaciones de referencia	6	por ejemplo add x5, x6, x7 = 007302b3, lui x6, 0x80000 = 80000337, sw x5, 4(x6) = 00532223
li, etiquetas, .equ y -D	9	partición alta/baja con parte baja negativa, li de 1 o 2 palabras, saltos hacia adelante, .equ con expresiones,

Grupo	Casos	Qué verifica
		redefinición con `-D`, advertencia de shifts
Errores detectados	13	cada tipo de error de la sección 5.7, incluyendo el reporte simultáneo de varios errores

5.9 Listado de salida

Con `-l`, el ensamblador genera un listado con la dirección, la palabra hexadecimal, el formato y los campos en binario de cada instrucción y la línea de origen. Las pseudo-instrucciones de dos palabras aparecen en dos líneas consecutivas. El listado se usó para revisar manualmente las codificaciones de la sección 5.6.

6. Reproducción

```
# Regenerar game.hex desde game.s
python asm/asm.py sw/game.s -o sw/game.hex

# Ensamblar, sintetizar, rutear y programar (Windows)
build

# Equivalente en Linux/macOS
make
```

`build` encadena `asm.py`, Yosys (`synth_ice40 -top game_top`), nextpnr (`--hx1k --package vq100 --freq 25`), `icepack` e `iceprog`, y se detiene si algún paso falla.

7. Conclusiones

Resultados. El juego realiza la secuencia con una sola modificación funcional de hardware, el contador de ciclos en `0x8000_000C`. Ocupa 184 palabras de 512, el diseño cierra temporización a 25 MHz con 92 % de las celdas lógicas, y el ensamblador propio reproduce byte a byte los `.hex` oficiales y el ensamblador GNU en los 32 casos de prueba.

Decisiones de diseño.

Decisión	Alternativa descartada	Motivo
Contador de 32 bits libre, leído por `lw`	contador de décimas en hardware	la unidad y las duraciones quedan en `game.s` y se cambian sin resintetizar
Plazos como diferencia `ahora − inicio`	comparar contra un valor absoluto	inmune al desborde cada 171,8 s
Dejar los shifts deshabilitados	barrel shifter en la ALU	no cabe: la FPGA ya está al 92 %

Decisión	Alternativa descartada	Motivo
Semilla = XOR con el contador en cada pulsación + LFSR de 32 bits	secuencia fija o LFSR sin semilla	la secuencia no se repite entre partidas
Índice con los bits 16 y 17 del LFSR	los bits bajos	los bits bajos son poco aleatorios con la cadencia del bucle de lectura
Botón de inicio al energizar	partida automática	sin él la primera ronda sería idéntica en cada arranque
Promedio sobre los valores mostrados, con redondeo al más cercano	promedio de ciclos exactos	el promedio es verificable a mano desde lo que se ve en pantalla
Todas las duraciones derivadas de `CICLOS_DECIMA`	constantes en ciclos escritas a mano	cambiar una constante y regenerar `game.hex` es una sola edición

Limitaciones conocidas. La medición tiene un retraso de detección de hasta 14 ciclos (560 ns) y se trunca a la décima; el tiempo mostrado se satura en 9,9 s; y la fase inicial dura algo más de 3 s por el antirrebote de 20 ms posterior.